

```
print("Computer Programming 2026\n")  
print("Department of Geography\n")  
print("16 April 2026")  
  
print("Week 8")
```

Array

- Group of consecutive cells
 - Representation of variable groups
 - With the same name AND THE SAME TYPE
- To refer to an ***element***, we must specify:
 - The name of the array
 - The ***position*** of the element (index)
- Syntax: *array_name*[*element_position*]
 - The first element has index **0**
 - The n-th element of the array **v** is **v[n-1]**

array name

	v[:,0]	v[:,1]	
v[0,0]	-45	87	v[0,1]
v[1,0]	6	4	v[1,1]
v[2,0]	0	9	v[2,1]
v[3,0]	72	-8	v[3,1]
v[4,0]	1543	-573	v[4,1]
v[5,0]	-89	-20	v[5,1]
v[6,0]	0	4	v[6,1]
v[7,0]	62	94	v[7,1]
v[8,0]	-3	22	v[8,1]
v[9,0]	1	0	v[9,1]
v[10,0]	6453	-551	v[10,1]
v[11,0]	78	10	v[11,1]

element position

A short summary...

- Array: collection of homogeneous data
 - Declare the type of its elements
 - Declare its dimension

```
data = np.zeros(DIM, int)
```

- How to access data in an array?

- Example of *writing*

```
for i in range(DIM):  
    data[i] = int(input(f"Insert the number (choice {i+1}): "))
```

- Example of *reading*

```
for i in range(DIM):  
    print(f"The number (choice {i+1}) is: {data[i]}")
```

Matrices: sum

- Two matrices A and B can be summed up if they have the same dimensions
- Their sum $A + B$ is defined as the matrix whose elements are obtained summing up the corresponding elements of A and B .
- Formally: $(A + B)_{i,j} := A_{i,j} + B_{i,j}$

$$\begin{bmatrix} 1 & 3 & 2 \\ 1 & 0 & 0 \\ 1 & 2 & 2 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 5 \\ 7 & 5 & 0 \\ 2 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1+0 & 3+0 & 2+5 \\ 1+7 & 0+5 & 0+0 \\ 1+2 & 2+1 & 2+1 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 7 \\ 8 & 5 & 0 \\ 3 & 3 & 3 \end{bmatrix}.$$

$$\mathbf{A} + \mathbf{B}$$

Matrices: multiplication by a scalar

- It is an operation that, given a matrix A and a number c (called *scalar*), creates a new matrix cA , whose element is obtained by multiplying the corresponding element of A by c

$$(cA)_{ij} := cA_{ij}.$$

$$2 \begin{bmatrix} 1 & 8 & -3 \\ 4 & -2 & 5 \end{bmatrix} = \begin{bmatrix} 2 \times 1 & 2 \times 8 & 2 \times -3 \\ 2 \times 4 & 2 \times -2 & 2 \times 5 \end{bmatrix} = \begin{bmatrix} 2 & 16 & -6 \\ 8 & -4 & 10 \end{bmatrix}.$$

$$\mathbf{c} * \mathbf{A}$$

Matrices: multiplication between matrices

Row-by-column product

- The multiplication is defined only if the number of rows of B coincides with the number n of columns of A . The result is a matrix with the same number of rows of A and the same number of columns of B
- The element in position (i,j) is given by the sum:

$$C_{i,j} := A_{i,1}B_{1,j} + A_{i,2}B_{2,j} + \cdots + A_{i,n}B_{n,j}.$$

Matrices: multiplication between matrices

$$\begin{bmatrix} 1 & 1 & 2 \\ 0 & 1 & -3 \end{bmatrix} \times \begin{bmatrix} 1 & 1 & 1 \\ 2 & 5 & 1 \\ 0 & -2 & 1 \end{bmatrix} =$$

$$\begin{bmatrix} C_{11} & C_{12} & C_{13} \\ C_{21} & C_{22} & C_{23} \end{bmatrix} = \begin{bmatrix} 3 & 2 & 4 \\ 2 & 11 & -2 \end{bmatrix}$$

$$C_{11} = [(1 \times 1) + (1 \times 2) + (2 \times 0)] = 3$$

$$C_{12} = [(1 \times 1) + (1 \times 5) + (2 \times -2)] = 2$$

$$C_{13} = [(1 \times 1) + (1 \times 1) + (2 \times 1)] = 4$$

$$C_{21} = [(0 \times 1) + (1 \times 2) + (-3 \times 0)] = 2$$

$$C_{22} = [(0 \times 1) + (1 \times 5) + (-3 \times -2)] = 11$$

$$C_{23} = [(0 \times 1) + (1 \times 1) + (-3 \times 1)] = -2$$



A * B

np.matmul(A, B)

A @ B

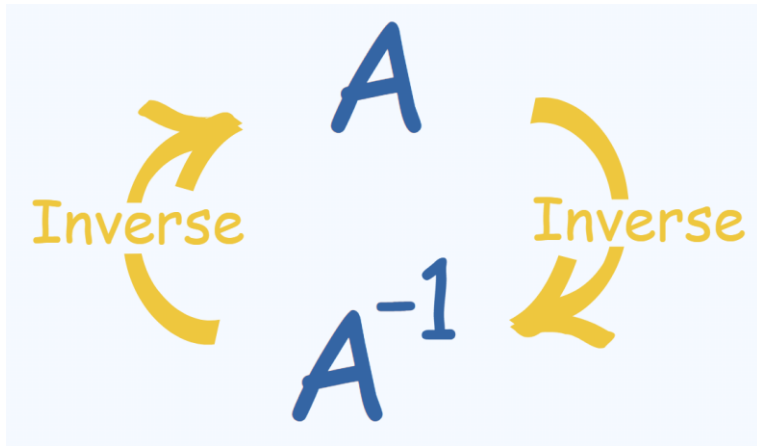
Matrices: transpose of a matrix

$$A = \begin{bmatrix} 7 & 2 & 9 & 1 \\ 2 & -1 & 3 & 0 \\ -5 & 3 & 0 & 11 \end{bmatrix} \quad B = \begin{bmatrix} 1 & 10 & -2 \\ 3 & -5 & 7 \\ 4 & 2 & -3 \end{bmatrix} \quad C = [1 \quad -1 \quad 7 \quad 8 \quad 3]$$

$$A^T = \begin{bmatrix} 7 & 2 & -5 \\ 2 & -1 & 3 \\ 9 & 3 & 0 \\ 1 & 0 & 11 \end{bmatrix} \quad B^T = \begin{bmatrix} 1 & 3 & 4 \\ 10 & -5 & 2 \\ -2 & 7 & -3 \end{bmatrix} \quad C^T = \begin{bmatrix} 1 \\ -1 \\ 7 \\ 8 \\ 3 \end{bmatrix}$$

A.T
B.T
C.T

Matrices: inverse of a matrix



`np.linalg.inv(A)`

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

$$\begin{aligned} \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix}^{-1} &= \frac{1}{4 \times 6 - 7 \times 2} \begin{bmatrix} 6 & -7 \\ -2 & 4 \end{bmatrix} \\ &= \frac{1}{10} \begin{bmatrix} 6 & -7 \\ -2 & 4 \end{bmatrix} \\ &= \begin{bmatrix} 0.6 & -0.7 \\ -0.2 & 0.4 \end{bmatrix} \end{aligned}$$

Matrices: elements' operations

	<code>np.sum(A)</code>
Sum of matrix elements	<code>np.sum(A, axis=0)</code> <code>np.sum(A, axis=1)</code>
	<code>np.mean(A)</code>
Mean of matrix elements	<code>np.mean(A, axis=0)</code> <code>np.mean(A, axis=1)</code>

Matrices: elements' operations

Minimum value among the matrix elements

`np.min(A)`

`np.min(A, axis=0)`

`np.min(A, axis=1)`

`np.argmin(A)`

`np.argmin(A, axis=0)`

`np.argmin(A, axis=1)`

Maximum value among the matrix elements

`np.max(A)`

`np.max(A, axis=0)`

`np.max(A, axis=1)`

`np.argmax(A)`

`np.argmax(A, axis=0)`

`np.argmax(A, axis=1)`

Exercise

- Write a program that, *after acquiring a cube of characters*, *calculates* how many 'a' are included in the cube.
 - The cube dimension is: 3x2x4
- Sub-problems
 - P0: Create an empty cube
 - P1: Populate the cube with characters
 - P2: Count the 'a'

Exercise

- P0: Create an empty cube (dimension: 3x2x4)

```
dim1 = 3
```

```
dim2 = 2
```

```
dim3 = 4
```

```
cube = np.zeros((dim1, dim2, dim3), str)
```

Exercise

- P1: Populate the cube with characters

```
for i in range(dim1):  
    for j in range(dim2):  
        for k in range(dim3):  
            cube[i,j,k] = input(f"Insert the character in position {i,j,k}: ")
```

Exercise

- P2: Count the 'a'

```
count = 0
for i in range(dim1):
    for j in range(dim2):
        for k in range(dim3):
            if cube[i,j,k] == 'a':
                count += 1
```

Exercise

- P2: Count the 'a'

```
values, counts = np.unique(cube, return_counts=True)  
dict(zip(values, counts))
```

```
np.sum(cube == "a")
```


A FEW NOTES...

A few notes...

- Create an array:

```
myarray = np.array([1, 2, 3, 4, 5])  
myarray = np.array((1, 2, 3, 4, 5))
```
- 0-D array:

```
myarray = np.array(42)
```
- 1-D array:

```
myarray = np.array([1, 2, 3, 4, 5])
```
- 2-D array:

```
myarray = np.array([[1, 2, 3], [4, 5, 6]])
```
- 3-D array:

```
myarray = np.array([[[1, 2, 3], [4, 5, 6]],  
                    [[1, 2, 3], [4, 5, 6]]])
```
- Check the number of dimensions:

```
print(myarray.ndim)  
print(myarray.shape)
```

A few notes...

- Print the first element of the array:

```
arr = np.array([1, 2, 3, 4])  
print(arr[0])
```

- Print the second element of the array:

```
arr = np.array([1, 2, 3, 4])  
print(arr[1])
```

- Select the third and fourth elements of the array and sum them up:

```
arr = np.array([1, 2, 3, 4])  
print(arr[2] + arr[3])
```

A few notes...

- Access the element on the first row, second column:

```
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])  
print(arr[0, 1])
```

- Access the element on the second row, fifth column:

```
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])  
print(arr[1, 4])
```

- Access the third element on the third dimension, in correspondence of the second column and the first row:

```
arr=np.array([[[1,2,3], [4,5,6]], [[7,8,9], [10,11,12]]])  
print(arr[0, 1, 2])
```

A few notes...

- Print the last element from the second dimension:

```
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])  
print(arr[1, -1])
```

- Slice elements from index 1 to index 4:

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])  
print(arr[1:5])
```

- Slice elements from index 4 to the end of the array:

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])  
print(arr[4:])
```

- Slice elements from the beginning to index 4 (not included):

```
print(arr[:4])
```

A few notes...

- Slice from index 3 (from the end) to index 1 (excluded, from the end):

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])  
print(arr[-3:-1])
```

- Return every other element from index 1 to index 5 (excluded):

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])  
print(arr[1:5:2])
```

- Slice the matrix from the second to the fourth column:

```
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])  
print(arr[:, 1:4])
```

A few notes...

- Check the data type of an array:

```
arr = np.array([1, 2, 3, 4])  
print(arr.dtype)
```

- Change the data type of an array from float to integer:

```
arr = np.array([1.1, 2.1, 3.1])  
arr2 = arr.astype(int)
```

- Change the data type of an array from integer to boolean:

```
arr = np.array([1, 0, 3])  
arr2 = arr.astype(bool)
```

- Copy an array

```
x = arr.copy()
```

A few notes...

- Convert a 1-D array into a 2-D array:

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])  
arr2 = arr.reshape(4, 3)
```

- Convert a 1-D array into a 3-D array:

```
arr2 = arr.reshape(2, 3, 2)
```

- Convert a 1-D array into a 3-D array (unknown dimension):

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])  
arr2 = arr.reshape(2, 2, -1)
```

- Flatten an array into a 1-D array:

```
arr = np.array([[1, 2, 3], [4, 5, 6]])  
arr2 = arr.reshape(-1)
```


A few notes...

- Join two arrays:

```
arr1 = np.array([1, 2, 3])  
arr2 = np.array([4, 5, 6])  
arr = np.concatenate((arr1, arr2))
```

- Join two arrays by stacking the columns together:

```
arr1 = np.array([[1, 2], [3, 4]])  
arr2 = np.array([[5, 6], [7, 8]])  
arr = np.concatenate((arr1, arr2), axis=1)
```

- Split an array in three parts:

```
arr = np.array([1, 2, 3, 4, 5, 6])  
arr2 = np.array_split(arr, 3)
```

A few notes...

- Split a 2-D array into three 2-D arrays by separating the columns:

```
arr=np.array([[1,2,3],[4,5,6],[7,8,9],[10,11,12],[13,14,15],[16,17,18]])  
arr2 = np.array_split(arr, 3, axis=1)
```

- Find the indexes where the value is 4:

```
arr = np.array([1, 2, 3, 4, 5, 4, 4])  
x = np.where(arr == 4)
```

- Find the indexes where the values are even:

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])  
arr2 = np.where(arr % 2 == 0)
```

A few notes...

- Find the indexes where the values are odd:

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])  
arr2 = np.where(arr % 2 == 1)
```

- Sort the array:

```
arr = np.array([3, 2, 0, 1])  
arr2 = np.sort(arr)
```

- Sort the array alphabetically:

```
arr = np.array(["banana", "cherry", "apple"])  
arr2 = np.sort(arr)
```

- Sort a 2-D array:

```
arr = np.array([[3, 2, 4], [5, 0, 1]])  
arr2 = np.sort(arr)
```

np.argsort(arr)

A few notes...

- Filter an array with a boolean list:

```
arr = np.array([41, 42, 43, 44])  
x = [True, False, True, False]  
arr2 = arr[x]
```
- Filter an array to keep only values greater than 42:

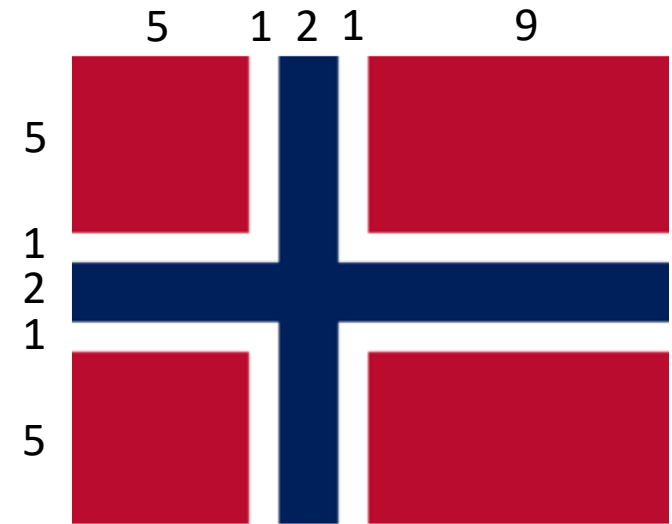
```
arr = np.array([41, 42, 43, 44])  
filter = arr > 42  
arr2 = arr[filter]
```
- Filter an array to keep only even elements:

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])  
filter = arr % 2 == 0  
arr2 = arr[filter]
```


Exercise

Create a 2D matrix representing the following flag
(assume white = 0, red = 1, blue = 2):

```
flag = np.ones((14, 18), int)
flag[:, 5:9] = 0
flag[5:9, :] = 0
flag[:, 6:8] = 2
flag[6:8, :] = 2
```



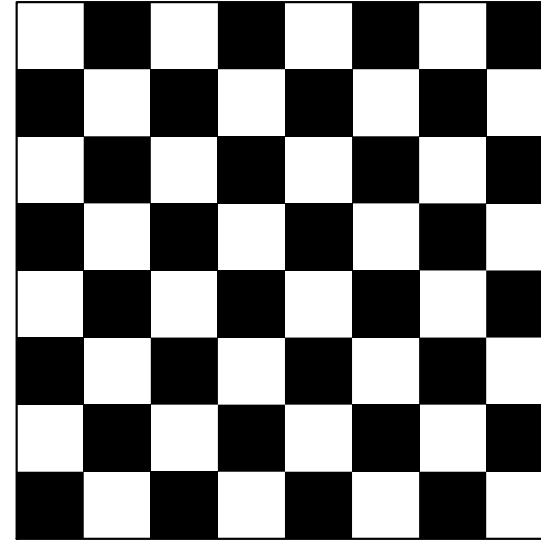
Exercise

Create the following chessboard:

- ❖ Model the board as a matrix
- ❖ black cells are represented as value 0
- ❖ white cells are represented as value 1.

```
board = np.zeros([8, 8], int)

for i in range(8):
    for j in range(0, 8, 2):
        if i % 2 == 0:
            board[i,j] = 1
        else:
            board[i,j+1] = 1
```



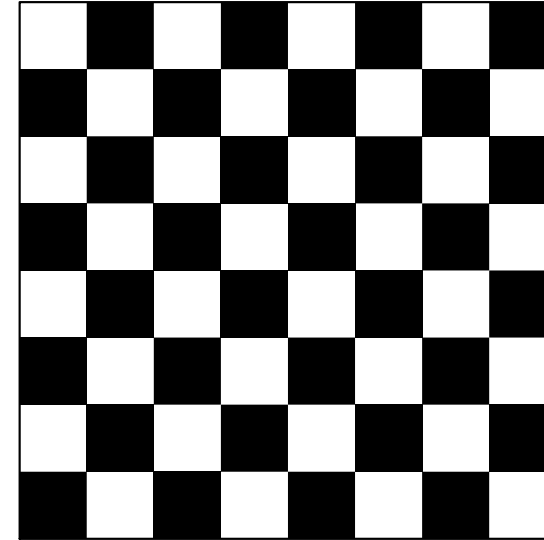
Exercise

Create the following chessboard:

- ❖ Model the board as a matrix
- ❖ black cells are represented as value 0
- ❖ white cells are represented as value 1.

```
board = np.zeros([8, 8], int)

for i in range(8):
    if i % 2 == 0:
        board[i, 0:8:2] = 1
    else:
        board[i, 1:8:2] = 1
```



```
import matplotlib.pyplot as plt
plt.imshow(board, cmap="gray")
plt.show()
```


STRINGS

Exercise

Write a program that, after acquiring a lower-case letter from keyboard, prints out on screen the equivalent upper-case letter

- Problem
 - Given an inserted character from keyboard, to be able to apply our transformation, we have to first verify that the type of character is what we expect.
- Solution
 - Define a pool of valid characters
 - Verify if the inserted character belongs to one of the valid characters

Pseudocode

- The pool of allowed characters
 {a, b, c, ..., z}

1. Ask for inserting a character
2. **If** the inserted character is correct
3. **Then print** on screen its upper-case version
4. **Else print** on screen a message of error

character-32

Pseudocode

- The pool of allowed characters
 $\{a, b, c, \dots, z\}$
- The inserted *character* must be:
 - $= > a$
 - $< = z$

Exercise

Write a program that, after acquiring a lower-case letter from keyboard, prints out on screen the equivalent upper-case letter

```
character = input("Insert a lower-case letter: ")

if character >= "a" and character <= "z":
    print("\nThe inserted letter is", character)
    print("Its upper-case representation is", chr(ord(character) - 32))
else:
    print("\nERROR: The inserted character is not correct")
```

Exercise

Write a program that, after acquiring a lower-case letter from keyboard, prints out on screen the equivalent upper-case letter

```
character = input("Insert a lower-case letter: ")

if character >= "a" and character <= "z" and len(character) == 1:
    print("\nThe inserted letter is", character)
    print("Its upper-case representation is", chr(ord(character) - 32))
else:
    print("\nERROR: The inserted character is not correct")
```

Exercise

- Write a program that, given an arbitrarily chosen lower and upper bound, prints out on screen all the characters included in such interval, together with the corresponding ASCII code

```
a = int(input("Choose the starting ASCII code: "))
b = int(input("Choose the ending ASCII code: "))

for i in range(a, b+1):
    print(f"\nThe character is {chr(i)} (its ASCII code is {i})")
```


7-bit ASCII Code Table

<u>Rightmost Four Bits</u>	<u>Leftmost Three Bits</u>							
	000	001	010	011	100	101	110	111
0000	NUL	DLE	Space	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

How many objects in the ASCII set?

- 32 non-printing control codes
 - DEL (delete) and SPACE (space) characters
 - 26 upper-case letters + 26 lower-case letters
 - 10 numbers
 - 32 punctuation symbols
-
- The ASCII code utilizes 7 bit, so it can represent maximum $2^7 = 128$ characters
 - There is also an “extended ASCII” on 8 bit ($2^8 = 256$ characters)

Are you sure it is really ASCII?



Unicode

- Unicode is a standard with the goal to cover all possible characters in the world (can hold up to 1,114,112 characters)
- One advantage of Unicode over other possible sets is that the first 256 code points are identical to ASCII

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F
0000																																
0020		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
0040	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
0060	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	{		}	~	
0080	€		,	f	„	...	†	‡	^	‰	Š	<	Œ		Ž			‘	’	“	”	•	—	~	™	š	>	œ		ž	ÿ	
00A0		ı	¢	£	¤	¥	¦	§	¨	©	ª	«	¬		®	¯	°	±	²	³	´	µ	¶	·	,	¹	º	»	¼	½	¾	¿
00C0	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
00E0	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ
0100	Ā	ā	Ă	ă	Ą	ą	Ć	ć	Ĉ	ĉ	Č	č	Ď	ď	Đ	đ	Ē	ē	Ĕ	ĕ	Ė	ė	Ę	ę	Ě	ě	Ĝ	ĝ	Ğ	ğ		
0120	Ġ	ġ	Ģ	ģ	Ĥ	ĥ	Ħ	ħ	Ĩ	ĩ	Ĳ	ĳ	Ĳ	ĳ	Ĳ	ĳ	Ĳ	ĳ	Ĳ	ĳ	Ĳ	ĳ	Ĳ	ĳ	Ĳ	ĳ	Ĳ	ĳ	Ĳ	ĳ	Ĳ	ĳ
0140	Ł	ł	Ń	ń	Ņ	ņ	Ň	ň	Ħ	ħ	Ŋ	ŋ	Ō	ō	Ŏ	ö	Ų	ų	Ŵ	ŵ	Ŷ	ŷ	Ÿ	Ž	ž	Ž	ž	Ž	ž	Ž	ž	
0160	Š	š	Ţ	ţ	Ť	ť	Ŧ	ŧ	Ũ	ũ	Ū	ū	Ŭ	ŭ	Ů	ů	Ű	ű	Ų	ų	Ŵ	ŵ	Ŷ	ŷ	Ÿ	Ž	ž	Ž	ž	Ž	ž	
0180	Ɓ	ƀ	Ɓ	ƀ	Ɓ	ƀ	Ɓ	ƀ	Ɓ	ƀ	Ɓ	ƀ	Ɓ	ƀ	Ɓ	ƀ	Ɓ	ƀ	Ɓ	ƀ	Ɓ	ƀ	Ɓ	ƀ	Ɓ	ƀ	Ɓ	ƀ	Ɓ	ƀ	Ɓ	ƀ
01A0	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ
01C0	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ
01E0	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ
0200	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ
0220	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ
0240	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ	Ɔ	ɔ

Sentence... collection of characters

- What is a sentence?
 - A collection of characters
- How do I represent it?
 - A collection of many variables!



supercalifragilisticexpialidocious

Strings

- **Strings** can be thought of as vectors of characters
- They represent “characters to be read sequentially”

- Definition of a string:

```
mystring = "word"
```

- **mystring** has 4 elements:

w	o	r	d
---	---	---	---

Exercise: Change a character

- Write a program that, given an inserted string, changes one (or more) character(s) of that string

```
string1 = input("Insert your name: ")
```

```
string2 = string1
```

```
string2[2] = "u"
```


Exercise: Change a character

- Write a program that, given an inserted string, changes one (or more) character(s) of that string

```
string1 = input("Insert your name: ")
```

```
string2 = string1
```

```
list2 = list(string2)  
list2[2] = "u"  
string2 = str(list2)
```

Exercise: Change a character

- Write a program that, given an inserted string, changes one (or more) character(s) of that string

```
string1 = input("Insert your name: ")
```

```
string2 = string1
```

```
list2 = list(string2)  
list2[2] = "u"  
string2 = "".join(list2)
```

A FEW NOTES...

A few notes...

- Assign string to a variable: `a = "Hello"`
- Assign a multi-line string to a variable: `a = """I really
really really
like strings"""`
- Print the second character: `a = "Hello, World!"
print(a[1])`
- Get the length of a string:
`a = "Hello, World!"
print(len(a))`

A few notes...

- Check if “free” is present in the following text:

```
txt = "The best things in life are free!"  
if "free" in txt:  
    print("Yes, 'free' is present")
```

- Check if “expensive” is NOT present in the following text:

```
txt = "The best things in life are free!"  
if "expensive" not in txt:  
    print("No, 'expensive' is not present")
```

- Get the characters from the third one to the fifth one

```
b = "Hello, World!"  
print(b[2:5])
```

A few notes...

- Get the first five characters:

```
b = "Hello, World!"  
print(b[:5])
```

- Get the characters from the third one to the end:

```
b = "Hello, World!"  
print(b[2:])
```

- Get the characters from “o” to “l” (“orl”):

```
b = "Hello, World!"  
print(b[-5:-2])
```

A few notes...

- Return the string in upper case:

```
a = "Hello, World!"  
print(a.upper())
```

- Return the string in lower case:

```
a = "Hello, World!"  
print(a.lower())
```

- Remove the space before and/or after the actual text:

```
a = " Hello, World! "  
print(a.strip())
```

A few notes...

- Replace a string with another string:

```
a = "Hello, World!"  
print(a.replace("H", "J"))
```

- Return a list of strings, based on a separator:

```
a = "Hello, World!"  
print(a.split(","))
```

- Merge variable a with variable b into variable c:

```
a = "Hello"  
b = "World"  
c = a + b  
  
c = a + " " + b
```




See you
next week!